Web Scraping

Advanced Data Analysis with Python - Week 6

Table of contents

- How does the internet work?
- HTTP and the internet
- Introduction to HTML
- Scraping 101
- Scraping a static webpage
- Scraping a dynamic page
- Challenge
- Advanced scraping: Selenium and social networks

How does the Internet work?

- A network of networks: your device, your ISP, the backbone, a web server
- Every machine has an IP address
- Data travels in small packets
- Protocols such as TCP/IP and HTTP define the shared rules
- Loading a page: DNS lookup, connection, request, response, rendering
- A scraper does the same steps, but skips the rendering

HTTP and the internet

Request

```
GET /products?page=2 HTTP/1.1
Host: example.com
User-Agent: MyBot/1.0
Accept: text/html
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 5120
<html> ... </html>
```

- Methods: GET reads, POST sends data
- Headers such as User-Agent and Cookie describe the client
- Each request stands alone

HTTP methods and status codes

Methods

- GET: read a resource
- POST: send data
- PUT / PATCH: update
- DELETE: remove

Status codes

- 200: OK
- 301: moved, follow the redirect
- 403: forbidden
- 404: not found
- 429: too many requests
- 500: server error

Scraping 101

- Scraping is extracting data from web pages automatically
- Step 1, fetch: send the HTTP request
- Step 2, parse: turn the HTML into a tree
- Step 3, extract: select the fields you need
- Step 4, store: write CSV, JSON or a database

Rules and tools

Scrape politely

- Read robots.txt and the terms of service
- Prefer an official API
- Pause between requests
- Identify yourself
- Respect personal data (GDPR)

Python tools

- requests: fetch pages
- BeautifulSoup, lxml: parse HTML
- Selenium, Playwright: drive a browser
- Scrapy: large crawls